Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

2° CUATRIMESTRE
🎮🚀🛠️ ACTIVIDAD 8: PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN C++ — JUEGO DE NAVES EN CONSOLA (class Nave) - LPR 5° 3° A-B



Estudiante: Mikaela Batirola
Curso: 5° 3°
Profesores: Yamil Ganduglia y York Muñoz Mansilla

Repositorio Git Hub:
https://github.com/mikabatirola434-svg/juegodenaves

Repositorio Git Lab:
https://gitlab.com/mikabatirola434-svg/juego-de-naves

Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

CUESTIONARIO TÉCNICO DE CONTROL

1. **struct vs class:**

   ¿Cuál es la principal diferencia técnica entre un struct (visto en Act. 7) y una clase, especialmente respecto a la sección private y public?

   La principal diferencia entre **`struct`** y **`class`** es que, en un **`struct`**, los atributos y métodos son públicos por defecto, mientras que en una **`class`** son privados por defecto.

   Esto significa que en una clase hay que usar **`public`** para permitir el acceso desde otras partes del programa y **`private`** para proteger los datos.

2. **Encapsulamiento:**

   ¿Por qué es peligroso dejar la variable salud como pública? ¿Qué desastre podría causar un programador novato desde el main()?

   Es peligroso dejar la variable **`salud`** como pública porque cualquier parte del programa podría modificarla directamente, sin ningún control. Por ejemplo, un programador podría cambiar la salud del jugador a un número negativo o a un valor demasiado alto desde el **`main`**`()`. Con el encapsulamiento, la variable se mantiene privada y se controla cómo se modifica mediante métodos públicos.

3. **Constructor:**

   Explicá con tus palabras para qué sirve el bloque Nave(string _nombre, ...) dentro del código.

   El constructor sirve para darle los valores iniciales a un objeto cuando se crea. En la clase `Nave`, se usa para indicar en qué posición va a aparecer la nave y cuántas vidas va a tener el jugador al empezar el juego. En el código, el constructor es **`Nave`**`(int _x, int _y, int _vidas)`, donde **`_x`** y **`_y`** representan la posición de la nave, y **`_vidas`** indica la cantidad de vidas iniciales. Así, cuando se crea la nave del jugador, ya tiene esos datos definidos y está lista para empezar la partida.

4. **Desafío Funcionalidad:**

   Explicá con tus palabras qué funcionalidad elegiste agregar de las 15 opciones y en qué partes del código tuviste que insertarla para que funcione.

Elegí agregar el disparo doble, que se activa cuando el jugador llega a los 300 puntos. Para hacerlo, utilicé el sistema de score, que suma 10 puntos cada vez que se destruye un asteroide.

Después, modifiqué la parte del `main()` que detecta cuando se presiona la barra espaciadora. Si tenes menos de 300 puntos, la nave dispara una bala desde el centro, y cuando llegas a 300 puntos, dispara dos balas desde los costados.

Funcionalidad 11:

C: > Users > LENOVO > Downloads > funcionalidad_11_disparo_doble.cpp

```
1   // [!] PISTA: Requiere la funcionalidad 1 (Score).
2   // Modificar el if(tecla == ' ') en el main() para instanciar dos objetos Bala.
3
4   if (score < 300) {
5       // Disparo simple central
6       balas.push_back(new Bala(jugador.getX() + 2, jugador.getY() - 1));
7   } else {
8       // Disparo doble desde las alas
9       balas.push_back(new Bala(jugador.getX(), jugador.getY()));
10      balas.push_back(new Bala(jugador.getX() + 4, jugador.getY()));
11  }
```

Código que activa el disparo doble al alcanzar los 300 puntos.

Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

<u>Figura 1:</u> Captura de la consola durante la ejecución del juego, donde se observa el combate entre la nave del jugador y los asteroides.

